Deep Network Development

Autumn 2026, Practice #8

Viktor Varga
Department of Artificial Intelligence, ELTE IK

Software for neural networks













CPU only

CPU & GPU

Last week - PyTorch: the Tensor type

The `torch.Tensor` type

```
temps2d = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

- `temps2d.numel()` `24`
- `temps2d.ndim` `2`
- `temps2d.size()` `torch.Size([2, 12])`
- `temps2d.shape` `torch.Size([2, 12])`
- `temps2d.dtype` `torch.float32`
- `temps2d.itemsize` `4`
- `temps2d.stride()` `(12, 1)`
- `temps2d._base` `None`
- `temps2d.device` `device(type='cpu')`
- `temps2d.requires_grad` `False`
- `temps2d.grad` `None`
- `temps2d.grad_fn` `None`

The Titanic Survival Prediction Dataset from Kaggle



Natural Disasters

View more

Titanic-Dataset.csv (61.19 kB)

Detail Compact Column 12 of 12 columns

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803

Last week - Preprocessing steps (The absolute minimum)

Preprocessing steps (Last week: the absolute minimum)

- Handling missing values
  - Replacing missing values by zeros.
- Splitting the dataset into training / validation / test sets

Last week - PyTorch: `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

Last week - PyTorch: A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Last week - PyTorch: A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

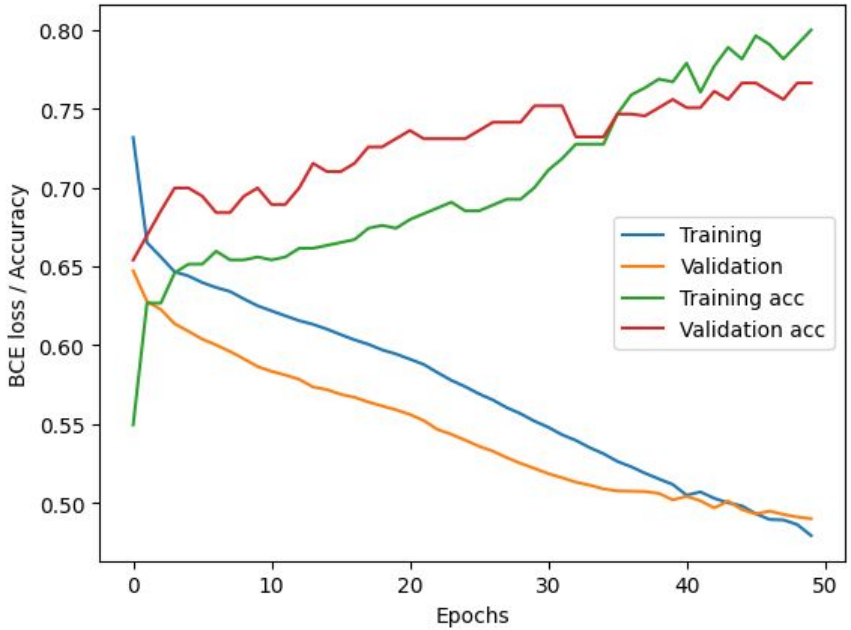
    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

In each epoch:

- Going through the **training set** once by mini-batch iteration and **training**.

- Going through the **validation set** once by mini-batch iteration and **evaluating**.

Last week - Loss curves



Preprocessing steps

Preprocessing steps

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	1	22	1	0	7.25	2
1	0	38	1	0	71.2833	0
3	0	26	0	0	7.925	2
1	0	35	1	0	53.1	2
3	1	35	0	0	8.05	2
3	1	(MISSING)	0	0	8.4583	1

Survived
0
1
0
1
0
0

Preprocessing steps

Preprocessing steps

- Handling missing values
  - Replace missing values with a constant

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	1	22	1	0	7.25	2
1	0	38	1	0	71.2833	0
3	0	26	0	0	7.925	2
1	0	35	1	0	53.1	2
3	1	35	0	0	8.05	2
3	1	(MISSING)	0	0	8.4583	1

Preprocessing steps

Preprocessing steps

- Handling missing values
  - Replace missing values with a constant
  - Creating new binary flag variables

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Age valid?
3	1	22	1	0	7.25	2	1
1	0	38	1	0	71.2833	0	1
3	0	26	0	0	7.925	2	1
1	0	35	1	0	53.1	2	1
3	1	35	0	0	8.05	2	1
3	1	0	0	0	8.4583	1	0

Preprocessing steps

Preprocessing steps

- Handling missing values
  - Replace missing values with a constant
  - Creating new binary flag variables

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Age valid?
3	1	22	1	0	7.25	2	1
1	0	38	1	0	71.2833	0	1
3	0	26	0	0	7.925	2	1
1	0	35	1	0	53.1	2	1
3	1	35	0	0	8.05	2	1
3	1	0	0	0	8.4583	1	0

Are we training a model, or are we trying to use a trained model?

→ Different techniques may be ideal in the two cases.

Preprocessing steps

Preprocessing steps

- Conversion of categorical variables to one-hot encoding

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	1	22	1	0	7.25	2
1	0	38	1	0	71.2833	0
3	0	26	0	0	7.925	2
1	0	35	1	0	53.1	2
3	1	35	0	0	8.05	2
3	1	(MISSING)	0	0	8.4583	1



Pclass	Sex	Age	SibSp	Parch	Fare	Embarked0	Embarked1	Embarked2
3	1	22	1	0	7.25	0	0	1
1	0	38	1	0	71.2833	1	0	0
3	0	26	0	0	7.925	0	0	1
1	0	35	1	0	53.1	0	0	1
3	1	35	0	0	8.05	0	0	1
3	1	(MISSING)	0	0	8.4583	0	1	0

Preprocessing steps

Preprocessing steps

- Shuffling the data points randomly

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	1	22	1	0	7.25	2
1	0	38	1	0	71.2833	0
3	0	26	0	0	7.925	2
1	0	35	1	0	53.1	2
3	1	35	0	0	8.05	2
3	1	(MISSING)	0	0	8.4583	1

Survived
0
1
0
1
0
0



Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
1	0	38	1	0	71.2833	0
3	1	(MISSING)	0	0	8.4583	1
1	0	35	1	0	53.1	2
3	1	35	0	0	8.05	2
3	1	22	1	0	7.25	2
3	0	26	0	0	7.925	2

Survived
1
0
1
0
0
0

Preprocessing steps

Preprocessing steps

- Balancing the dataset / Resampling
  - Resampling the dataset

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	1	22	1	0	7.25	2
1	0	38	1	0	71.2833	0
3	0	26	0	0	7.925	2
1	0	35	1	0	53.1	2
3	1	35	0	0	8.05	2
3	1	(MISSING)	0	0	8.4583	1

Survived
0
1
0
1
0
0



Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	0	26	0	0	7.925	2
3	1	35	0	0	8.05	2
3	1	(MISSING)	0	0	8.4583	1
1	0	38	1	0	71.2833	0
1	0	35	1	0	53.1	2
1	0	38	1	0	71.2833	0

Survived
0
0
0
1
1
1

Preprocessing steps

Preprocessing steps

- Balancing the dataset / Resampling
  - Resampling the dataset
  - Weighted loss

```
loss_fn = nn.BCELoss(reduction=None)
```

Preprocessing steps

Preprocessing steps

- Splitting the dataset into training / validation / test sets

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	1	22	1	0	7.25	2
1	0	38	1	0	71.2833	0
3	0	26	0	0	7.925	2

Survived
0
1
0

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
1	0	35	1	0	53.1	2
3	1	35	0	0	8.05	2

Survived
1
0

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	1	(MISSING)	0	0	8.4583	1

Survived
0

Preprocessing steps

Preprocessing steps

- Feature / label scaling

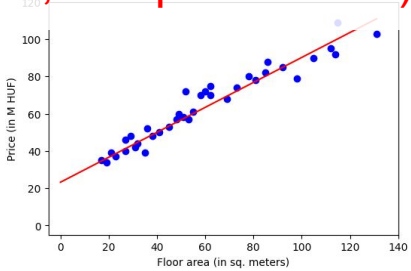
Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	1	22	1	0	7.25	2
1	0	38	1	0	71.2833	0
3	0	26	0	0	7.925	2
1	0	35	1	0	53.1	2
3	1	35	0	0	8.05	2
3	1	(MISSING)	0	0	8.4583	1



Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
1	1	0	1	0	0	1
0	0	1	1	0	1	0
1	0	0.25	0	0	0.0105	1
0	0	0.8125	1	0	0.716	1
1	1	0.8125	0	0	0.0125	1
1	1	(MISSING)	0	0	0.0188	0.5

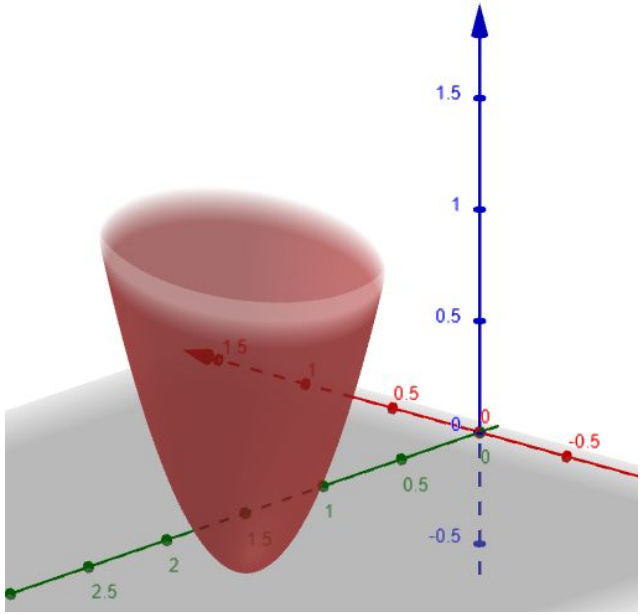
(Visualization: a single input variable, two parameters)

Feature scaling

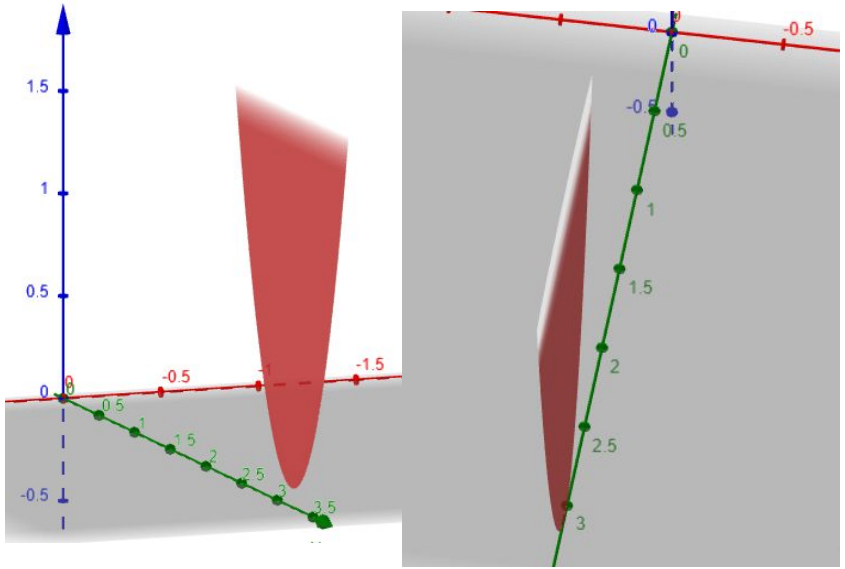


Problem:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{j=1}^{m} (\theta_1 x^{(j)} + \theta_0 - y^{(j)})^2$$



The loss function, as we wish to see it…



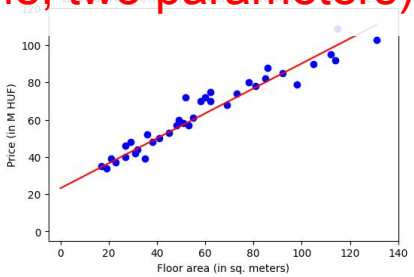
The loss function in reality

(Visualization: a single input variable, two parameters)

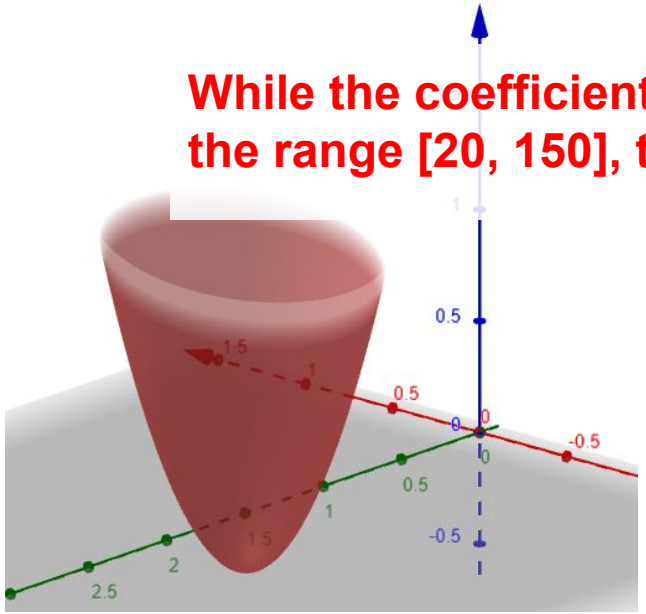
Feature scaling

Problem:

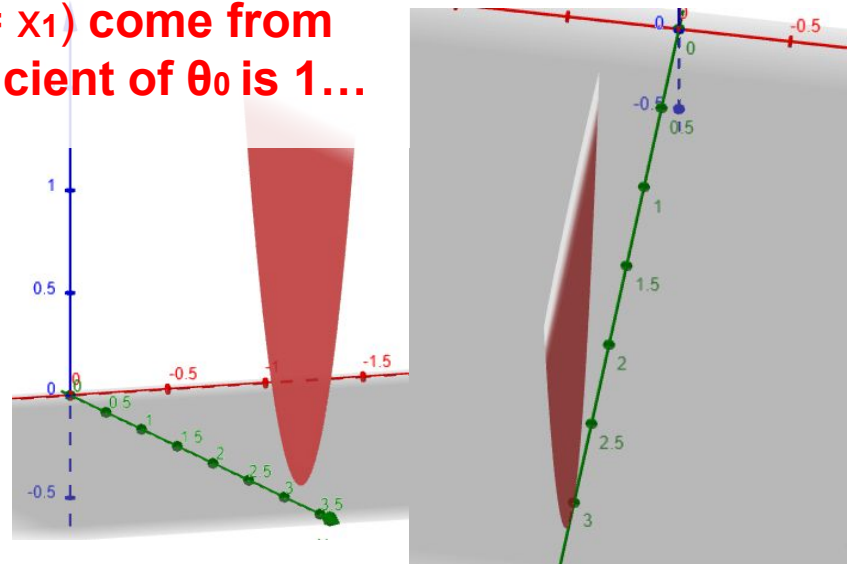
$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{j=1}^{m} (\theta_1 x^{(j)} + \theta_0 - y^{(j)})^2$$



While the coefficients of θ_1 (= x_1) come from the range [20, 150], the coefficient of θ_0 is 1…



The loss function, as we wish to see it…



The loss function in reality

Image sources: Jannes Klaas, kaggle.com

Feature scaling

Problem: Gradient descent does not work well on such surfaces...

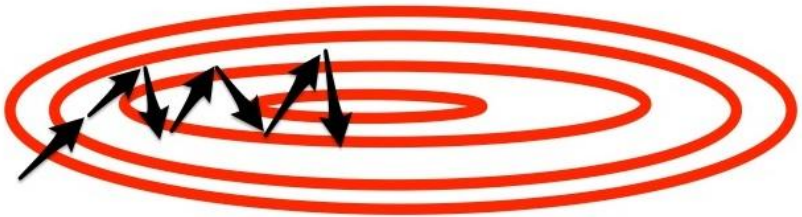
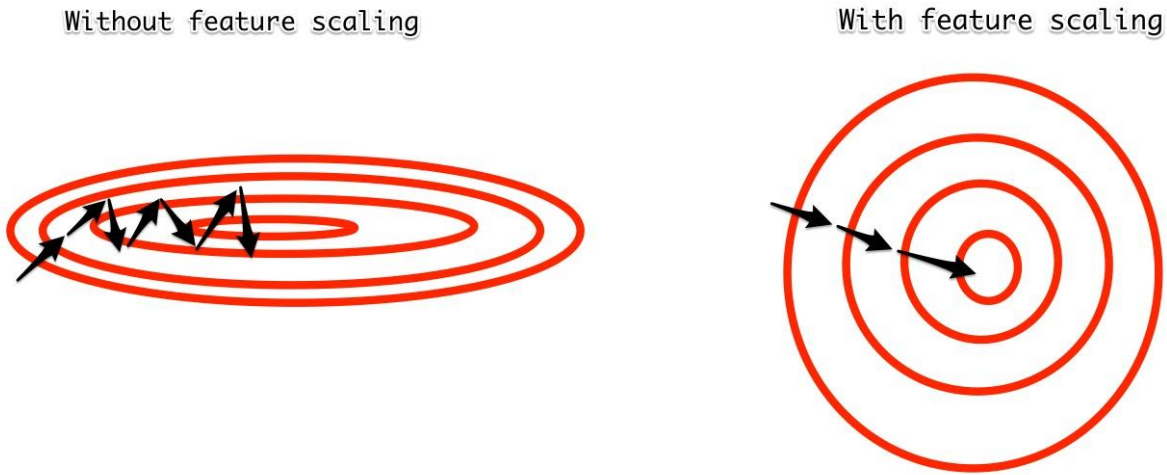


Image sources: Jannes Klaas, kaggle.com

Feature scaling

Feature scaling: Scaling input variables to the same order of magnitude



Feature scaling

Independently for each variable!

Feature scaling: Scaling input variables to the same order of magnitude

- **Min-max scaling** to the [0, 1] interval:

$$\forall i, j : x_i^{(j)} = \frac{x_i^{(j)} - min_j(\{x_i^{(j)}\})}{max_j(\{x_i^{(j)}\}) - min_j(\{x_i^{(j)}\})}$$

- **Standardization** (mean = 0, standard deviation = 1):

$$\forall i, j : x_i^{(j)} = \frac{x_i^{(j)} - \mu_j(\{x_i^{(j)}\})}{\sigma_j(\{x_i^{(j)}\})}$$

μ: mean
σ: standard deviation

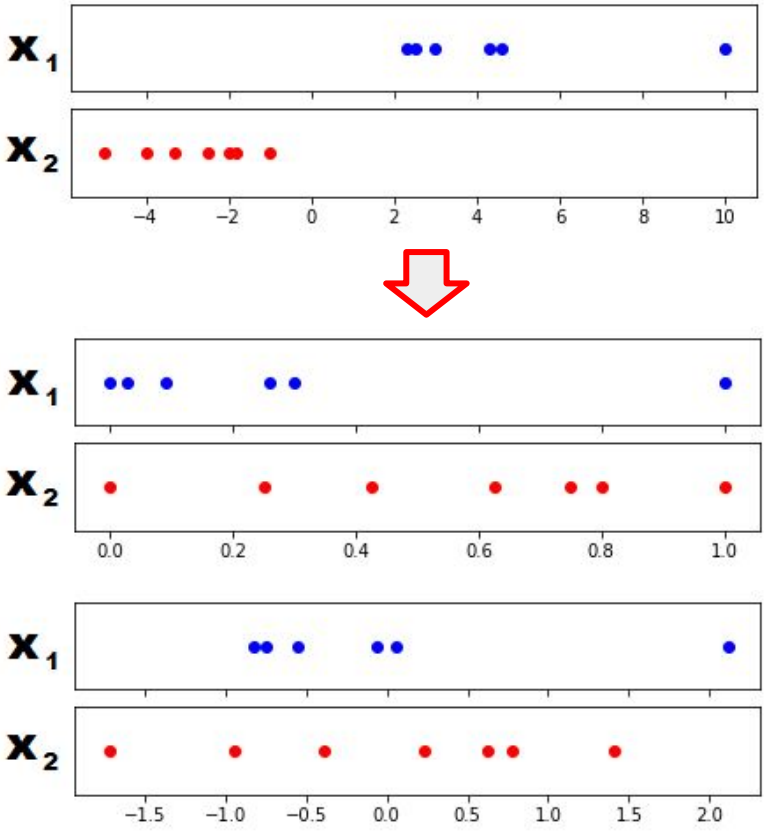
Feature scaling

Independently for each variable!

Feature scaling: Scaling input variables to the same order of magnitude

- **Min-max scaling** to the [0, 1] interval:

- **Standardization** (mean = 0, standard dev. = 1):



Feature scaling

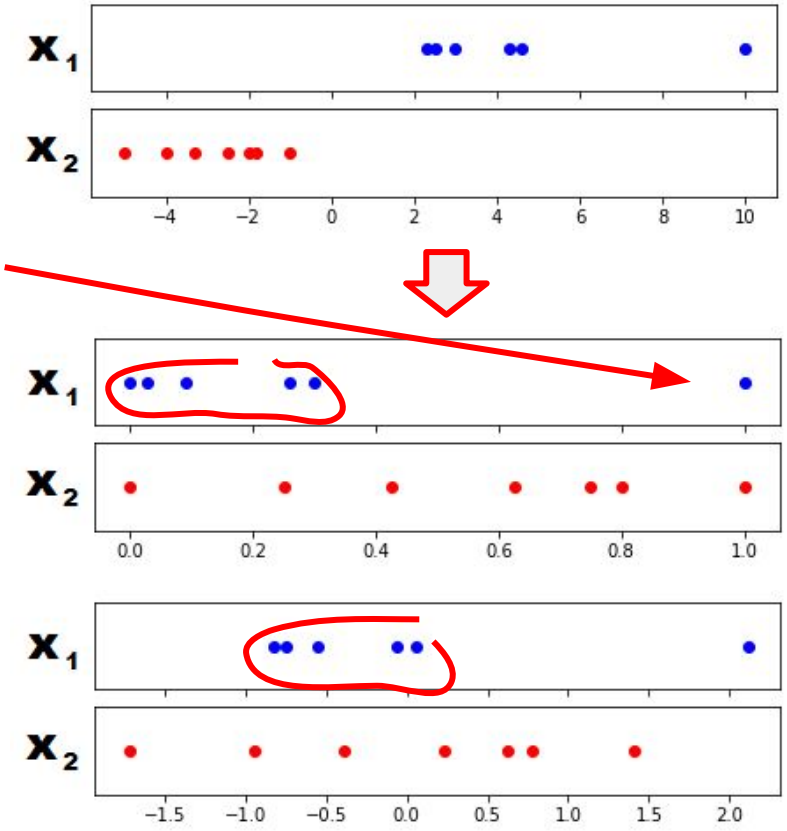
Independently for each variable!

Feature scaling: Scaling input variables to the same order of magnitude

Min-max scaling: More sensitive to outliers

- **Min-max scaling** to the [0, 1] interval:

- **Standardization** (mean = 0, standard dev. = 1):



Preprocessing steps

Preprocessing steps

- Feature / label scaling

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
3	1	22	1	0	7.25	2
1	0	38	1	0	71.2833	0
3	0	26	0	0	7.925	2
1	0	35	1	0	53.1	2
3	1	35	0	0	8.05	2
3	1	(MISSING)	0	0	8.4583	1

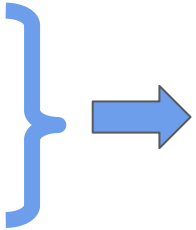


Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
1	1	0	1	0	0	1
0	0	1	1	0	1	0
1	0	0.25	0	0	0.0105	1
0	0	0.8125	1	0	0.716	1
1	1	0.8125	0	0	0.0125	1
1	1	(MISSING)	0	0	0.0188	0.5

Regression - Multiple label variables

Example task with **a single label variable:**

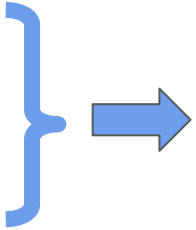
x_1: Weight of a patient
x_2: Age of a patient
x_3: Sex of a patient



y: Cholesterol levels of the patient

Example task with **multiple label variables:**

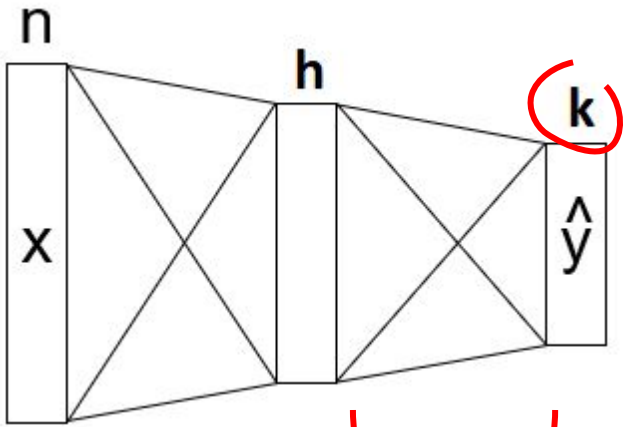
x_1: Weight of a patient
x_2: Age of a patient
x_3: Sex of a patient



y_1: Cholesterol levels of the patient
y_2: Blood sugar levels of the patient

MLP - Multiple label variables

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$



$W_1 \in \mathbb{R}^{h \times n}$ $W_2 \in \mathbb{R}^{k \times h}$

$b_1 \in \mathbb{R}^{h}$ $b_2 \in \mathbb{R}^{k}$

Let y be a vector, similarly to x!

$$\Theta = \{W_1, b_1, W_2, b_2\}$$

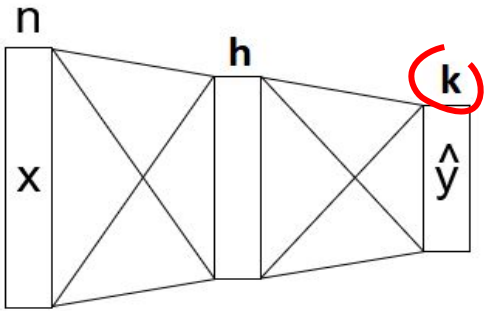
MLP - Multiple label variables

Regression

y hat and **y** are vectors

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$

Since in regression our labels can contain arbitrary numbers, g_2 is typically an identity function (i.e., it can be omitted).



MLP - Multiple label variables

Regression

y hat and **y** are vectors

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$

Loss: We average the squared loss over the elements of the label vector.

Until now (y was a scalar): $J(\Theta) = \frac{1}{2m} \sum_{j=1}^{m} (\hat{y}^{(j)} - y^{(j)})^2$

y is a vector: $J(\Theta) = \frac{1}{2mk} \sum_{j=1}^{m} ||\hat{y}^{(j)} - y^{(j)}||_2^2 = \frac{1}{2mk} \sum_{j=1}^{m} \sum_{i=1}^{k} (\hat{y}_i^{(j)} - y_i^{(j)})^2$

MLP - Multiple label variables

Regression

y hat and **y** are vectors

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$

Loss: We average the squared loss over the elements of the label vector.

Until now (y was a scalar): $J(\Theta) = \frac{1}{2m}\sum_{j=1}^{m}(\hat{y}^{(j)} - y^{(j)})^2$

y is a vector: $J(\Theta) = \frac{1}{2mk}\sum_{j=1}^{m}||\hat{y}^{(j)} - y^{(j)}||_2^2 = \frac{1}{2mk}\sum_{j=1}^{m}\sum_{i=1}^{k}(\hat{y}_i^{(j)} - y_i^{(j)})^2$

MSE as before**, but now we also average over the elements of the label vector…**

MLP - Multiple label variables

Regression

y hat and **y** are vectors

$$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2) = \hat{y} \approx y$$

Loss: We average the squared loss over the elements of the label vector.

Until now (y was a scalar): $J(\Theta) = \frac{1}{2m} \sum_{j=1}^{m} (\hat{y}^{(j)} - y^{(j)})^2$

… or, if the label variables are not on the same scale:

- **Scale the labels similarly to feature scaling**
- **Use weighted averaging**

y is a vector: $J(\Theta) = \frac{1}{2mk} \sum_{j=1}^{m} ||\hat{y}^{(j)} - y^{(j)}||_2^2 = \frac{1}{2mk} \sum_{j=1}^{m} \sum_{i=1}^{k} (\hat{y}_i^{(j)} - y_i^{(j)})^2$

```
loss_fn = nn.MSELoss(reduction=None)
```

Preprocessing steps

Preprocessing steps

- Handling missing values
- Conversion of categorical variables to one-hot encoding
- Shuffling the data points randomly
- Balancing the dataset / Resampling
- Splitting the dataset into training / validation / test sets
- Feature / label scaling

The steps to take and their order may vary…
(The steps were not combined in the figures.)

Mini-batch iteration with **tensor slicing**

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
    ...
```

Mini-batch iteration with **tensor slicing**

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
    ...
```

Tensor(current_batch_size, n_input_vars)

Tensor(current_batch_size, 1)

(in case of binary classification)

Mini-batch iteration with **a custom iterator (class)**

```
class MinibatchIterator:

  def __init__(self, input_split, labels_split, batch_size):
    self.input_split = input_split                    # input_split: (n_samples, n_input_vars)
    self.labels_split = labels_split                  # labels_split: (n_samples,)
    self.batch_size = batch_size                      # batch_size: int
    self.offset_idx = 0

  def __iter__(self):
    return self

  def __next__(self):
    if self.offset_idx < self.input_split.shape[0]:
      batch_xs = self.input_split[self.offset_idx:self.offset_idx+self.batch_size, :]
      batch_ys = self.labels_split[self.offset_idx:self.offset_idx+self.batch_size, None]
      self.offset_idx += self.batch_size
      return batch_xs, batch_ys
    else:
      raise StopIteration()
```

Mini-batch iteration with **a custom iterator (class)**

```
class MinibatchIterator:

  def __init__(self, input_split, labels_split, batch_size):
    self.input_split = input_split                    # input_split: (n_samples, n_input_vars)
    self.labels_split = labels_split                  # labels_split: (n_samples,)
    self.batch_size = batch_size                      # batch_size: int
    self.offset_idx = 0

  def __iter__(self):
    return self

  def __next__(self):
    if self.offset_idx < self.input_split.shape[0]:
      batch_xs = self.input_split[self.offset_idx:self.offset_idx+self.batch_size, :]
      batch_ys = self.labels_split[self.offset_idx:self.offset_idx+self.batch_size, None]
      self.offset_idx += self.batch_size
      return batch_xs, batch_ys
    else:
      raise StopIteration()
```

Tensor(current_batch_size, n_input_vars), Tensor(current_batch_size, 1)

(in case of binary classification)

Mini-batch iteration with **a custom iterator (class)**

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    minibatch_iterator = MinibatchIterator(inputs_train_t, labels_train_t, batch_size)
    for train_batch_xs, train_batch_ys in minibatch_iterator:

        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())

    ...
```

Mini-batch iteration with **a custom iterator (class)**

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    minibatch_iterator = MinibatchIterator(inputs_train_t, labels_train_t, batch_size)
    for train_batch_xs, train_batch_ys in minibatch_iterator:

        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())

    ...
```

Tensor(current_batch_size, n_input_vars)

Tensor(current_batch_size, 1)

(in case of binary classification)

Mini-batch iteration with **a custom iterator (generator)**

```
def minibatch_generator(input_split, labels_split, batch_size):          # input_split: (n_samples, n_input_vars)
                                                                         # labels_split: (n_samples,)
                                                                         # batch_size: int

    assert input_split.shape[:1] == labels_split.shape
    for offset_idx in range(0, input_split.shape[0], batch_size):
        batch_xs = input_split[offset_idx:offset_idx+batch_size, :]
        batch_ys = labels_split[offset_idx:offset_idx+batch_size, None]
        yield batch_xs, batch_ys
```

Mini-batch iteration with **a custom iterator (generator)**

```
def minibatch_generator(input_split, labels_split, batch_size):          # input_split: (n_samples, n_input_vars)
                                                                         # labels_split: (n_samples,)
                                                                         # batch_size: int
    assert input_split.shape[:1] == labels_split.shape
    for offset_idx in range(0, input_split.shape[0], batch_size):
        batch_xs = input_split[offset_idx:offset_idx+batch_size, :]
        batch_ys = labels_split[offset_idx:offset_idx+batch_size, None]
        yield batch_xs, batch_ys
```



Tensor(current_batch_size, n_input_vars), Tensor(current_batch_size, 1)

(in case of binary classification)

Mini-batch iteration with **a custom iterator (generator)**

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for train_batch_xs, train_batch_ys in minibatch_generator(inputs_train_t, labels_train_t, batch_size):

        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())

    ...
```

Mini-batch iteration with **a custom iterator (generator)**

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for train_batch_xs, train_batch_ys in minibatch_generator(inputs_train_t, labels_train_t, batch_size):

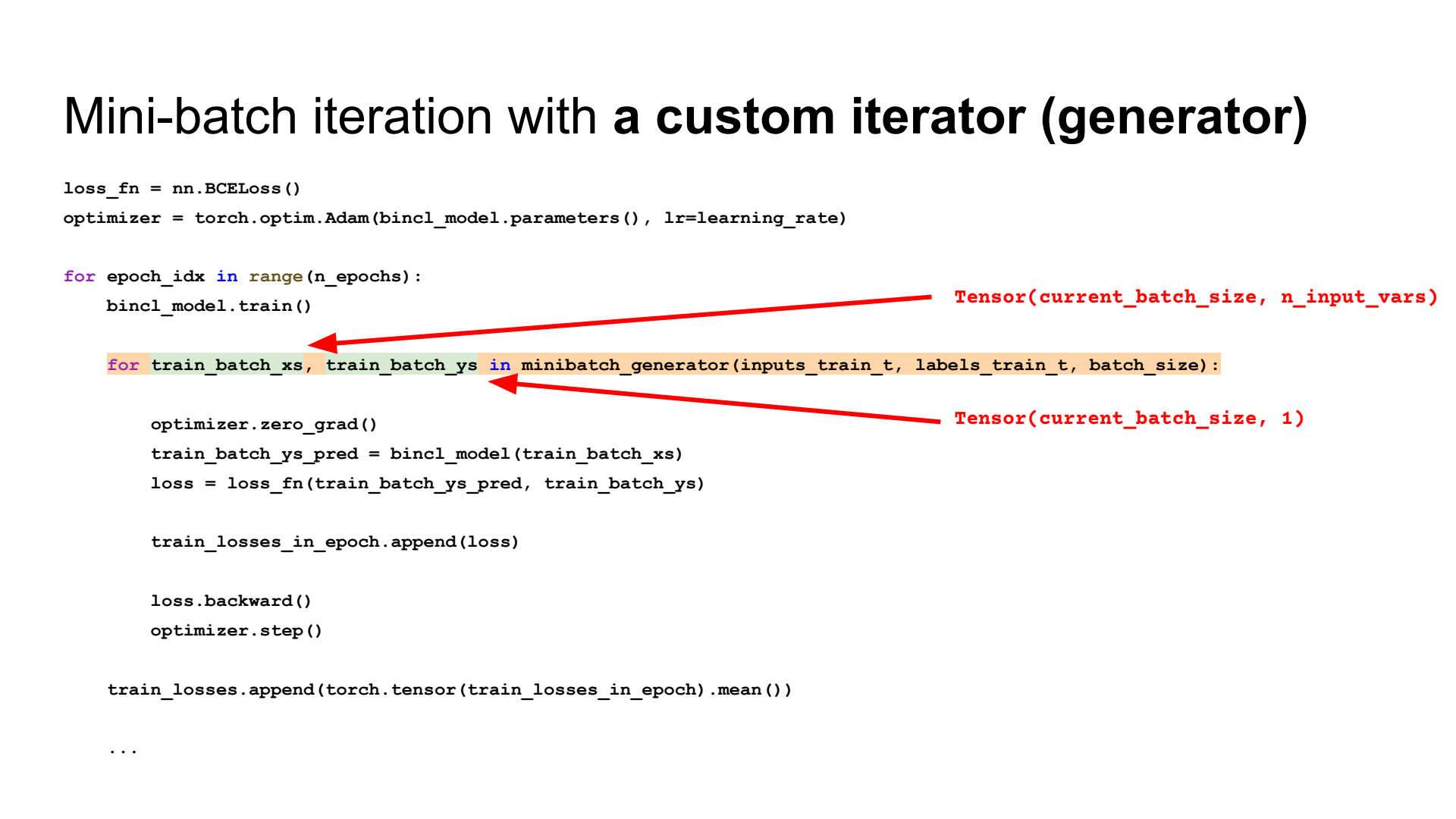
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())

    ...
```



Mini-batch iteration with **Dataset / DataLoader**

```
class BinaryClassificationDataset(torch.utils.data.Dataset):
    def __init__(self, input_split, labels_split):
        self.xs = torch.tensor(input_split, dtype=torch.float32)
        self.ys = torch.tensor(labels_split, dtype=torch.float32)
        # (true) labels dtype:
        #     Binary classification: float32 (a single probability per data point)

    def __len__(self):
        return self.xs.shape[0]

    def __getitem__(self, idx):
        return self.xs[idx, :], self.ys[idx, None]


train_dataset = BinaryClassificationDataset(inputs_train, labels_train)
dataloader_train = DataLoader(train_dataset, batch_size=32, shuffle=True)
```

Mini-batch iteration with **Dataset / DataLoader**

```
class BinaryClassificationDataset(torch.utils.data.Dataset):
    def __init__(self, input_split, labels_split):
        self.xs = torch.tensor(input_split, dtype=torch.float32)
        self.ys = torch.tensor(labels_split, dtype=torch.float32)
        # (true) labels dtype:
        #     Binary classification: float32 (a single probability per data point)

    def __len__(self):
        return self.xs.shape[0]

    def __getitem__(self, idx):
        return self.xs[idx, :], self.ys[idx, None]
```

(in case of binary classification)

← `Tensor(n_input_vars,), Tensor(1,)`

```
train_dataset = BinaryClassificationDataset(inputs_train, labels_train)
dataloader_train = DataLoader(train_dataset, batch_size=32, shuffle=True)
```

The iterator of `dataloader_train` will yield: `tuple(Tensor(current_batch_size, n_input_vars), Tensor(current_batch_size, 1))`

Mini-batch iteration with **Dataset / DataLoader**

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for train_batch_xs, train_batch_ys in dataloader_train:

        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())

    ...
```

Mini-batch iteration with **Dataset / DataLoader**

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for train_batch_xs, train_batch_ys in dataloader_train:

        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())

    ...
```

Tensor(current_batch_size, n_input_vars)

Tensor(current_batch_size, 1)

(in case of binary classification)